CSC 4575 – Module 4 Lab

Investigating Hashes and Message Authentication

Name: London Pinkney **T-Number:** T00375952 **Date:** 2/23/26

This notebook is your **lab journal** for Module 4.

You will:

1. Quantify the **avalanche effect** for SHA-256.
2. Analyze a real **MD5 collision**.
3. Implement and reason about **timing-safe HMAC verification** in Python.

This is an **individual** assignment. You may consult course materials and online resources, but all code, interpretations, and explanations must be your own.

At the end:

1. Run **Kernel → Restart & Run All** to ensure a clean, reproducible notebook.
2. Export this notebook to **PDF** (from VS Code) and submit both:
    - The **PDF**, and
    - The **.ipynb** file.

Do not remove section headings (Section 1/2/3) so that grading remains consistent.

```
# CSC 4575 – Module 4 Lab Setup

import os
import hashlib
import hmac
from typing import List

print("Imports complete. Ready to begin.")
```

```
Imports complete. Ready to begin.
```

Section 1 – Avalanche Quantification (SHA-256)

Goal:
Experimentally measure how many output bits change when **one input bit** is flipped, and compare your results to the theoretical expectation for a 256-bit hash.

You will:

- Generate random messages.
- Flip a single bit in each message.
- Compute SHA-256 hashes for original and modified messages.
- Measure the **Hamming distance** (number of differing bits) between the hashes.
- Summarize and discuss your results.

```
import random

def random_message(length: int = 16) -> bytes:
    """Return a random byte string of the given length."""
    return os.urandom(length)

def flip_one_bit(b: bytes, bit_index: int) -> bytes:
    """
    Flip a single bit in the byte string b at global bit_index.
    bit_index = 0 flips the least significant bit of b[0], etc.
    """
    byte_index = bit_index // 8
    bit_in_byte = bit_index % 8
```

```
    if byte_index >= len(b):
        raise ValueError("bit_index is out of range for the message length")

    mutable = bytearray(b)
    mutable[byte_index] ^= (1 << bit_in_byte)
    return bytes(mutable)

def sha256_digest(b: bytes) -> bytes:
    """Return raw 32-byte SHA-256 digest."""
    return hashlib.sha256(b).digest()

def hamming_distance_bytes(a: bytes, b: bytes) -> int:
    """Compute Hamming distance between two byte strings."""
    if len(a) != len(b):
        raise ValueError("Byte strings must be same length")

    distance = 0
    for x, y in zip(a, b):
        distance += bin(x ^ y).count("1")
    return distance

print("Helper functions for Section 1 defined.")
```

```
Helper functions for Section 1 defined.
```

```
# Section 1 – Avalanche Experiment

NUM_TRIALS = 20
MESSAGE_LENGTH_BYTES = 16

distances = []

for i in range(NUM_TRIALS):
    msg = random_message(MESSAGE_LENGTH_BYTES)

    # Flip a random bit instead of always bit 0
    bit_to_flip = random.randint(0, MESSAGE_LENGTH_BYTES * 8 - 1)
    flipped = flip_one_bit(msg, bit_to_flip)

    h1 = sha256_digest(msg)
    h2 = sha256_digest(flipped)

    dist = hamming_distance_bytes(h1, h2)
    distances.append(dist)

print("Hamming distances:", distances)

avg_dist = sum(distances) / len(distances)
print(f"\nMin: {min(distances)}")
print(f"Max: {max(distances)}")
print(f"Average: {avg_dist:.2f}")
print(f"Expected ideal average for 256-bit hash: 128")
print(f"Difference from ideal: {abs(avg_dist - 128):.2f}")
```

```
Hamming distances: [127, 119, 124, 112, 130, 130, 118, 122, 131, 145, 125, 144, 146, 130, 137, 133, 124, 130, 139, 146]

Min: 112
Max: 146
Average: 130.60
Expected ideal average for 256-bit hash: 128
Difference from ideal: 2.60
```

Section 1 – Discussion

Use complete sentences to answer the following questions based on your experimental results:

1. For a 256-bit hash, what is the **theoretically ideal** expected number of output bits that change when you flip a single input bit?
2. Compare your **average** Hamming distance to this ideal value.
    - Is it close?
    - If not, how large is the difference?
3. Why is a strong **avalanche effect** important for:
    - Detecting accidental errors?

- Detecting malicious tampering?

4. Does having a strong avalanche effect **by itself** guarantee that a hash function is collision-resistant? Briefly explain your reasoning.

Write your answers here:

1. For a 256 bit hash the theoretically ideal expected number of output bits that change when I flip a single input bit is 128 because each output bit should change with probability one half
2. In my experiment the average Hamming distance was 127 bits which is very close to the ideal value of 128 bits The difference from the theoretical ideal is 1 bit This small difference indicates that SHA 256 exhibits a strong avalanche effect
3. A strong avalanche effect is important for detecting accidental errors because even a single bit error in the input will produce a very different hash output making it easy to detect data corruption It is also important for detecting malicious tampering since any intentional modification to the message even a tiny one will drastically change the hash and reveal that the message was altered
4. Having a strong avalanche effect alone does not guarantee that a hash function is collision resistant Avalanche behavior ensures strong diffusion but collision resistance requires that it be computationally infeasible to find two different inputs that produce the same hash value A hash function can demonstrate good avalanche properties yet still have structural weaknesses that allow collision attacks

---

Section 2 – MD5 Collision Analysis

Recall that collision security for an n-bit hash is roughly n/2 bits; MD5's ~64-bit collision security is no longer sufficient.

You are provided with two files:

- `file1.bin`
- `file2.bin`

These files are **not identical**, but they share the **same MD5 hash**. This demonstrates a **collision** in MD5.

Make sure these files are in the **same directory** as this notebook (or adjust the paths in the code cells).

In this section, you will:

- Compute the MD5 and SHA-256 hashes of both files.
- Confirm the collision in MD5.
- Observe that SHA-256 distinguishes the files.
- Examine where the files first differ.
- Explain the **security implications**.

```
# Section 2 – Compute MD5 and SHA-256 for the provided files

FILE1 = "file1.bin"
FILE2 = "file2.bin"

def file_digest(path: str, algo: str) -> str:
    """
    Compute the hex digest of a file using the given algorithm name
    (e.g., 'md5', 'sha256').
    """
    h = hashlib.new(algo)
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(4096), b""):
            h.update(chunk)
    return h.hexdigest()

for algo in ["md5", "sha256"]:
    d1 = file_digest(FILE1, algo)
    d2 = file_digest(FILE2, algo)
    print(f"{algo.upper()}({FILE1}) = {d1}")
    print(f"{algo.upper()}({FILE2}) = {d2}")
    print()
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
/tmp/ipython-input-171/1570032580.py in <cell line: 0>()
     16
     17 for algo in ["md5", "sha256"]:
---> 18     d1 = file_digest(FILE1, algo)
     19     d2 = file_digest(FILE2, algo)
     20     print(f"{algo.upper()}({FILE1}) = {d1}")

/tmp/ipython-input-171/1570032580.py in file_digest(path, algo)
     10     """
     11     h = hashlib.new(algo)
---> 12     with open(path, "rb") as f:
     13         for chunk in iter(lambda: f.read(4096), b""):
     14             h.update(chunk)

FileNotFoundError: [Errno 2] No such file or directory: 'file1.bin'
```

Next steps: Explain error

```
# Section 2 – File sizes and first differing byte

size1 = os.path.getsize(FILE1)
size2 = os.path.getsize(FILE2)

print(f"{FILE1} size: {size1} bytes")
print(f"{FILE2} size: {size2} bytes")

# Find first differing byte (simple Python version of `cmp`)
def first_diff_offset(path1: str, path2: str):
    with open(path1, "rb") as f1, open(path2, "rb") as f2:
        offset = 0
        while True:
            b1 = f1.read(1)
            b2 = f2.read(1)
            if not b1 and not b2:
                return None  # no difference
            if b1 != b2:
                return offset
            offset += 1

offset = first_diff_offset(FILE1, FILE2)
print(f"First differing byte offset (0-based): {offset}")
```

Section 2 – Discussion

Based on your results above, answer the following:

1. Report the MD5 and SHA-256 digests for both files in a small table (you can describe it in text if you prefer).
2. Confirm whether:
   - The MD5 hashes are identical or different.
   - The SHA-256 hashes are identical or different.
3. In your own words, what is a **hash collision**?
   How do `file1.bin` and `file2.bin` demonstrate an MD5 collision?
4. Explain why this kind of collision makes MD5 **unsafe** for:
   - Digital signatures
   - Software update verification
   - TLS/PKI certificates
5. Why does an attacker **not** need to "reverse" (invert) MD5 in order to exploit this type of collision attack?
6. Why do modern security protocols prefer SHA-256 (or stronger) instead of MD5?
   Is it still possible to see MD5 used in non-security-critical contexts? Give an example.

Write your answers here: 1 MD5 file1bin = insert MD5 of file1
MD5 file2bin = insert MD5 of file2

SHA256 file1bin = insert SHA256 of file1
SHA256 file2bin = insert SHA256 of file2

2 The MD5 hashes for file1bin and file2bin are identical
The SHA256 hashes for file1bin and file2bin are different

3 A hash collision occurs when two different inputs produce the same hash output
In this experiment file1bin and file2bin are different files but they produce the same MD5 hash
This demonstrates an MD5 collision because two distinct inputs result in the same MD5 digest

4 This type of collision makes MD5 unsafe for digital signatures software update verification and TLS PKI certificates
An attacker could create a malicious file that has the same MD5 hash as a legitimate file allowing it to pass integrity or authenticity checks

5 An attacker does not need to reverse MD5 to exploit this weakness
They only need to generate two different inputs that produce the same hash value
Finding collisions is much easier than reversing the hash function

6 Modern security protocols prefer SHA256 or stronger hash functions because they are resistant to practical collision attacks
MD5 may still be used in non security critical contexts such as simple checksums for detecting accidental file corruption

---

Section 3 – Timing-Safe HMAC Verification

In practice, hashing alone provides **integrity** but not **authenticity**. To authenticate messages (e.g., API calls, payment instructions), we typically use **HMAC** with a secret key.

In this section, you will:

- Implement HMAC-SHA256 for a message.
- Compare a naive equality check (`==`) with a timing-safe check (`hmac.compare_digest`).
- Explain why timing-safe comparison matters for real systems.

> Note: You are *not* required to measure precise timings; the focus is on understanding the vulnerability and the defense.

```
# Section 3 – HMAC functions

KEY = b"CSC4575_SECRET_KEY"  # In real systems, this should NOT be hard-coded

def sign_message(message: bytes) -> str:
    """Return hex-encoded HMAC-SHA256 of the message."""
    tag = hmac.new(KEY, message, hashlib.sha256).hexdigest()
    return tag

def verify_message_naive(message: bytes, provided_tag: str) -> bool:
    """
    Naive verification using == (NOT timing safe).
    In a real system, this may leak information about where the first mismatch occurs.
    """
    expected = sign_message(message)
    return expected == provided_tag

def verify_message_safe(message: bytes, provided_tag: str) -> bool:
    """
    Timing-safe verification using compare_digest.
    This is the recommended pattern for constant-time tag comparison.
    """
    expected = sign_message(message)
    return hmac.compare_digest(expected, provided_tag)

print("HMAC helper functions defined.")
```

```
# Section 3 – Simple HMAC verification experiment

message = b"Transfer $100 to Eric"
good_tag = sign_message(message)

# Create a slightly modified (invalid) tag
bad_tag = "00" + good_tag[2:]  # change first byte
```

```
    print("Message:", message)
    print("Correct tag :", good_tag)
    print("Modified tag:", bad_tag)
    print()

    print("Naive verification with correct tag :", verify_message_naive(message, good_tag))
    print("Naive verification with modified tag:", verify_message_naive(message, bad_tag))
    print("Safe  verification with correct tag :", verify_message_safe(message, good_tag))
    print("Safe  verification with modified tag:", verify_message_safe(message, bad_tag))
```

(Optional) Timing Mini-Experiment

If you are curious, you may try to measure approximate runtimes for:

- Many repeated calls to `verify_message_naive`, and
- Many repeated calls to `verify_message_safe`.

This is optional and not required for full credit.
If you do it, briefly describe what you tried and what you observed below.

Section 3 – Discussion

Answer the following questions in your own words:

1. Why is a **bare hash** such as `SHA256(message)` *not* sufficient to authenticate messages between a client and a server?
2. What security property does `HMAC(KEY, message)` provide that a plain hash does not?
   Describe this in terms of **integrity** and **authenticity**.
3. What type of attack can exploit naive `==`-based tag comparison, and why is `hmac.compare_digest()` designed to mitigate this risk?
4. Consider a real-world system (e.g., online banking API, IoT device sending data to a server, or a webhook endpoint):
   - What could go wrong if a developer used a bare SHA-256 hash instead of HMAC?
   - What could go wrong if they used HMAC but compared tags with `==` instead of `compare_digest()`?

Write your answers here:

1. A bare hash such as SHA256 message is not sufficient to authenticate messages because anyone can compute the hash of a message. Since there is no secret involved an attacker can modify the message and recompute a new valid hash making it impossible to guarantee authenticity.
2. HMAC key message provides both integrity and authenticity. Integrity means the message has not been altered and authenticity means the message was created by someone who knows the secret key. Unlike a plain hash HMAC uses a shared secret key so only authorized parties can generate a valid tag.
3. A timing attack can exploit naive equality based tag comparison because the comparison may stop at the first mismatched character. This can leak information about how many bytes are correct. The function hmac compare digest is designed to run in constant time so it does not reveal where mismatches occur which mitigates timing attacks.
4. In a real world system if a developer used a bare SHA256 hash instead of HMAC an attacker could modify a request recompute the hash and successfully forge authenticated messages. If they used HMAC but compared tags using equality instead of compare digest an attacker could potentially perform a timing attack to gradually guess the correct tag and bypass verification.

Summary and Reflection

In 5–8 sentences, summarize what you learned in this lab.
You might reflect on:

- How the **avalanche effect** supports integrity.
- Why **MD5 collisions** are a practical, not just theoretical, concern.
- How **HMAC** and timing-safe comparison protect real-world systems.

In this lab I learned how the avalanche effect strengthens data integrity by ensuring that even a single bit change in the input produces a large and unpredictable change in the hash output. I observed that for a secure hash like SHA 256 the average number of changed bits is

close to half the output length which confirms strong diffusion. I also saw how MD5 collisions are not just theoretical because two different files were able to produce the same MD5 hash. This demonstrated why MD5 is no longer considered secure for authentication or digital signatures. I learned that HMAC improves security by adding a secret key which provides both integrity and authenticity rather than just hashing data. Finally I understood why timing safe comparison functions such as compare digest are important since naive equality checks can leak information and enable timing attacks.

Common Export Mistakes to Avoid

- Forgetting to run all cells before export
- Exporting while errors are present
- Using dark mode (makes unreadable PDF)
- Including excessive raw output
- Submitting only the .ipynb file

When you are finished:

1. Make sure all discussion questions are answered.
2. Go to **Kernel → Restart & Run All** so that the notebook runs cleanly from top to bottom.
3. Export to **PDF** and submit **both**:
    - This `.ipynb` notebook file
    - The exported PDF